



FACULTY OF COMPUTING
UNIVERSITY OF
SRI JAYEWARDENEPURA

ShortStay – Online Marketplace for Short - Term Accommodations

K.U.E. JAYAWEERA

GROUP - 02

Registration number: FC110620

Index number: FC213008

Name of the Supervisor:

Mr. K.D. Tharindu Rangana

July 2026

**This dissertation is submitted in partial fulfillment of the requirement of the Degree of
Bachelor of Computing (Honors) in Information Systems of the
Faculty of Computing, University of Sri Jayewardenepura.**

Declaration

“I certify that this dissertation does not incorporate, without acknowledgment, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text. I also hereby give consent for my dissertation, if accepted, to be made available for photocopying and for interlibrary loans, and for the title and abstract to be made available to outside organizations.

Signature of Candidate: Date :

Name of Candidate:

Countersigned by:

Signature of Supervisor(s)/Advisor(s): Date :

Name(s) of Supervisor(s)/Advisor(s):

Abstract

The rapid growth of tourism and the increasing demand for short-term accommodations have created new opportunities for property owners and travelers. However, many local property owners face challenges when using existing international accommodation platforms due to complexity, limited local support, and the lack of proper verification mechanisms. Therefore, this project introduces ShortStay, a web-based online marketplace designed to provide a simple, reliable, and locally focused platform for short-term accommodation services.

The main objective of ShortStay is to connect guests with verified hosts while providing a secure and efficient accommodation booking experience. The system consists of five main user roles: Guest, Host, Admin, Accountant, and Verifier. Guests can search, filter, and book available accommodations, while hosts can manage property listings and booking requests. The Admin manages overall system operations, the Accountant manages financial transactions, payments, and related records, while the Verifier inspects properties and ensures that accommodations meet the required quality standards before approval.

The system was developed using an agile software development approach to support iterative development and continuous improvement. The frontend was implemented using React.js, the backend was developed using Node.js with Express.js, and MySQL was used as the database management system. UML diagrams, database design, and role-based architecture were used to ensure system clarity, scalability, and maintainability.

The completed ShortStay system provides essential functionalities including property management, accommodation booking, payment management, property verification, and user review mechanisms. The platform helps local property owners generate additional income opportunities while providing guests with a trusted and convenient accommodation booking solution.

Keywords: Short-term accommodation, Online marketplace, Property management, Booking system, Verification system, Web application

Acknowledgment

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of our project, “ShortStay – Online Marketplace for Short-Term Accommodations.”

First and foremost, we would like to thank our project supervisor, Mr. K.D. Tharindu Rangana, for the valuable guidance, continuous support, and constructive feedback provided throughout the project. The advice and recommendations helped us improve our project idea, overcome challenges, and successfully complete this work.

We would also like to extend our appreciation to the lecturers and staff members of the Faculty of Computing, University of Sri Jayewardenepura, for providing the knowledge, resources, and academic environment necessary to complete this project successfully.

Our sincere appreciation goes to all team members for their commitment, cooperation, and teamwork throughout the requirement analysis, system design, development, testing, and documentation phases. The contribution of each member in their assigned responsibilities played an important role in achieving the objectives of this project.

Finally, we would like to thank our families, friends, and everyone who supported us with encouragement and assistance throughout the completion of this project.

Table of Contents

Declaration	ii
Abstract	iii
Acknowledgment	iv
Table of Contents	v
Table of Figures	viii
List of Tables	ix
List of Acronyms	x
Chapter 1 - INTRODUCTION	11
1.1 Motivation for the project.....	11
1.2 Objectives.....	11
1.3 Scope of the project.....	11
Chapter 2 - ANALYSIS	13
2.1 Current Manual method.....	13
2.2 Drawbacks of Existing method.....	13
2.3 Review of Similar systems.....	13
2.4.1 Functional requirements.....	13
2.4.2 Non-functional requirements.....	13
2.5 Process model.....	14
Chapter 3 - DESIGN	15
3.1 Design strategies.....	15
3.2 Justification for the selected design strategy.....	15
3.3 Architectural design.....	15
3.4 Object-Oriented Methodical approach.....	15
3.5 High-Level Use Case Diagram for the Proposed System.....	15
3.6 ER Diagram for the Proposed System.....	15

3.6 Class Diagram for the Proposed System.....	15
3.7 User Interface Designing for the System.....	15
Chapter 4 – IMPLEMENTATION	16
4.1 Introduction.....	16
4.2 Implementation Environment.....	16
4.3 System Developed Tools and Technologies.....	16
4.4 Major Code Segments.....	16
4.5 System module structure.....	16
4.6 Re-usable Components.....	17
Chapter 5 – EVALUATION	18
5.1 Introduction.....	18
5.2 System Testing.....	18
5.3 Testing Techniques.....	18
5.4 Testing Methodologies.....	18
5.5 System Test Plan.....	18
5.6 System Test Cases.....	18
5.7 User acceptance test.....	19
5.7 User Feedback.....	19
Chapter 6 – CONCLUSION	20
6.1 Introduction.....	20
6.2 Problems Encountered.....	20
6.3 Future Improvements.....	20
6.4 Lessons Learnt.....	20
References	21
Appendix A - System Manual	22
Appendix B - Design Documentation	23
Appendix C - User Manual	24

Appendix D - Management Reports	25
Appendix E - Test Results	26
Appendix F - Client Certificate	27
Glossary	28
Index	29

Table of Figures

List of Tables

List of Acronyms

Chapter 1 - INTRODUCTION

1.1 Motivation for the project

The tourism industry in Sri Lanka has shown continuous growth over the past decade, contributing significantly to the national economy. With increasing tourist arrivals and domestic travel demand, the need for short-term accommodation services has also increased. Many property owners, especially in local areas, are now interested in renting their spaces to earn additional income.

However, existing global platforms such as Airbnb and Booking.com present several challenges for local users. These platforms are often complex, less localized, and may include high service charges. In addition, property verification processes are not always transparent for users, which can reduce trust between hosts and guests.

Furthermore, many local property owners face difficulties in managing online listings due to lack of technical knowledge and platform complexity. On the other hand, travelers often struggle to find affordable, verified, and reliable accommodations through a simple system tailored to local needs.

To address these challenges, the ShortStay system is introduced as a web-based online marketplace for short-term accommodations. It aims to provide a simple, efficient, and trustworthy platform where guests can easily find accommodations and hosts can manage their properties effectively. The system also introduces structured roles such as Accountant and Verifier to improve financial transparency and property trustworthiness, which are often limited in existing systems.

Therefore, ShortStay is developed to bridge the gap between property owners and travelers by providing a localized, user-friendly, and secure accommodation booking platform.

1.2 Objectives

Main Objective

The main objective of this project is to develop a web-based short-term accommodation marketplace that enables secure interaction between guests and hosts while ensuring reliable payment handling and property verification.

Specific Objectives

- To develop a web platform for searching and booking short-term accommodations
- To allow hosts to create, manage, and update property listings

- To implement a secure authentication system for all users
- To introduce a Verifier role for property inspection and approval
- To introduce an Accountant role for managing payments, transactions, and financial records
- To enable administrators to manage users and overall system operations
- To improve trust and transparency in the short-term rental process
- To provide an easy-to-use interface suitable for local users and tourists

1.3 Scope of the project

The ShortStay system is designed as a web-based application that supports multiple user roles and core functionalities related to short-term accommodation management.

Included Scope

Guest

- User registration and login
- Searching and filtering accommodations
- Viewing property details
- Booking accommodations
- Making payments
- Writing reviews and ratings
- Viewing booking history

Host

- Property registration and listing creation
- Managing property details (price, images, availability)
- Managing booking requests
- Viewing earnings and booking status

Admin

- Managing all system users
- Approving or rejecting property listings
- Monitoring system activities
- Handling user complaints and system control

Accountant

- Managing payment transactions
- Handling host payouts
- Monitoring financial records
- Processing refunds and payment verification

Verifier

- Conducting property inspections
- Verifying property quality and safety standards
- Approving or rejecting property verification requests
- Maintaining inspection records

Excluded Scope

The following functionalities are not included in this project:

- Mobile application development
- Real-time chat between users
- Transportation or travel services
- Integration with external booking platforms
- Long-term rental management systems

1.4 Outline of the remaining chapters

This report consists of six chapters that describe the complete development process of the ShortStay – Online Marketplace for Short-Term Accommodations system.

Chapter 2 – Analysis

This chapter discusses the analysis phase of the ShortStay system. It explains the requirement gathering techniques used, analysis of existing accommodation platforms, identified limitations, and the requirements of the proposed system. Furthermore, relevant system analysis diagrams such as use case diagrams, class diagrams, sequence diagrams, and entity relationship diagrams are presented to provide a clear understanding of the system functionality and structure.

Chapter 3 – Design

This chapter presents the design phase of the proposed system based on the identified requirements. It describes the overall system architecture, database design, user interface design, and design approaches used during the development process. Relevant design diagrams and interface designs are included to demonstrate the structure and behavior of the ShortStay system.

Chapter 4 – Implementation

This chapter explains the implementation process of the ShortStay system. It discusses the development environment, technologies, frameworks, and tools used for implementation. The implementation of major system modules, including user management, property management, booking management, payment management, and property verification modules, are also described.

Chapter 5 – Evaluation

This chapter focuses on the testing and evaluation process carried out to ensure the quality and reliability of the ShortStay system. It explains the testing strategies, test

cases, testing results, and user acceptance testing performed to verify whether the implemented system satisfies the identified requirements.

Chapter 6 – Conclusion

This chapter provides the overall conclusion of the project by discussing the achievements, challenges encountered during development, lessons learned, and possible future improvements that can enhance the ShortStay system.

Chapter 2 - ANALYSIS

2.1 Current Manual method

Currently, many short-term accommodation arrangements in Sri Lanka are handled through informal and semi-digital methods. Property owners commonly advertise their accommodations through social media platforms, personal websites, online advertisements, or by sharing information among friends and communities. Guests who are interested in renting these properties usually contact property owners directly through phone calls or messaging applications to inquire about availability, pricing, facilities, and booking details.

The booking process often requires manual communication between guests and property owners. Guests must request available dates, negotiate prices, confirm accommodation details, and make payments through separately arranged methods such as bank transfers or cash payments. Property owners manually maintain booking records and update availability based on received requests.

Although some international platforms provide online accommodation booking services, many local property owners still depend on manual processes due to platform complexity, service charges, and lack of localized support. As a result, there is a requirement for a centralized system that simplifies accommodation management while improving trust between guests and hosts.

The current manual accommodation process can be summarized as follows:

1. The property owner advertises accommodation details.
2. Guest searches for available accommodations.
3. Guest contacts the property owner.
4. The owner provides details and confirms availability.
5. Guests complete payment through an agreed method.
6. The owner manually records the booking.

This process creates difficulties in managing accommodation availability, verifying properties, and maintaining transparent communication.

2.2 Drawbacks of Existing method

The current manual method of managing short-term accommodations has several limitations that affect both guests and property owners.

- Lack of Centralized Platform

Since accommodation details are distributed across different sources such as social media pages and advertisements, guests must spend additional time searching for suitable properties.

- Limited Property Verification

Many existing informal methods do not include a proper property verification process. Guests may face uncertainty regarding property quality, safety, and the accuracy of provided information.

- Manual Booking Management

Property owners often manage reservations manually, which can lead to:

- Double bookings
- Incorrect availability information
- Difficulty tracking booking history

- Payment Management Issues

Payments are commonly handled separately between guests and hosts. This creates difficulties in tracking transactions, handling refunds, and maintaining payment records.

- Limited Trust Between Users

Without a proper review, rating, and verification mechanism, guests may hesitate to book unfamiliar accommodations.

- Challenges for Local Users

International platforms may include complex procedures, high service fees, and limited consideration for local requirements, making them less convenient for some local hosts.

Therefore, a localized online platform is required to overcome these limitations and provide a more reliable accommodation management solution.

2.3 Review of Similar systems

Several existing online accommodation platforms were analyzed to identify their features, advantages, and limitations.

Airbnb

Airbnb is a global accommodation marketplace that connects hosts with guests. It allows property owners to publish listings and enables guests to search and reserve accommodations.

Main Features:

- Property listing management
- Search and filtering
- Online booking
- Reviews and ratings
- Payment handling

Limitations:

- High service charges
- Less focused on local requirements
- Complex processes for some new users
- Verification process is not directly controlled by local communities

Booking.com

Booking.com provides online reservation services for hotels and private accommodations worldwide.

Main Features:

- Large accommodation database
- Online reservations
- Customer reviews
- Price comparison

Limitations:

- Mainly focused on established accommodation providers
- Limited opportunities for small local property owners
- Less flexibility for local accommodation management

Agoda

Agoda is another global travel booking platform that provides accommodation reservations.

Main Features:

- Accommodation search
- Online booking
- Promotions and discounts
- Customer reviews

Limitations:

- Primarily targets commercial accommodation providers
- Limited support for small-scale local hosts

Comparison with ShortStay

ShortStay is designed to overcome these limitations by providing a locally focused accommodation marketplace.

ShortStay introduces:

- Localized user-friendly experience
- Property verification through the Verifier
- Payment monitoring through the Accountant
- Direct support for local hosts
- Role-based management through Admin control

2.4 Requirements of the system

The requirements of the ShortStay system were identified through analysis of the current accommodation process, user needs, and limitations of existing systems.

The system requirements are categorized into:

- Functional Requirements
- Non-functional Requirements

2.4.1 Functional requirements

Functional requirements define the services and features that the ShortStay system provides to users.

Guest Requirements

The system shall allow guests to:

- Register and log into the system
- Search accommodations
- Filter properties by location, price, and availability
- View property details
- Book accommodations
- Make payments
- View booking history
- Submit reviews and ratings

Host Requirements

The system shall allow hosts to:

- Create and manage property listings
- Upload property information and images
- Update availability and pricing
- Receive booking requests
- Manage reservations
- View earnings

Admin Requirements

The system shall allow administrators to:

- Manage all users
- Monitor system activities
- Approve or remove listings
- Manage complaints
- Generate reports

Accountant Requirements

The system shall allow accountants to:

- Monitor transactions
- Manage payment records
- Handle refunds

- Track host payments
- Generate financial reports

Verifier Requirements

The system shall allow verifiers to:

- View verification requests
- Inspect property details
- Submit verification results
- Approve or reject properties
- Maintain inspection records

2.4.2 Non-functional requirements

Security

The system should protect user information through secure authentication and authorization mechanisms.

Usability

The system should provide a simple and user-friendly interface for different user groups.

Performance

The system should respond quickly and efficiently handle user requests.

Reliability

The system should provide consistent service availability with minimal failures.

Maintainability

The system should be easy to update and modify in the future.

Scalability

The system should support increasing numbers of users and properties.

2.5 Process model

The ShortStay system was developed using an Agile Software Development Methodology.

Agile was selected because it supports iterative development, continuous improvement, and frequent feedback.

The development process was divided into several iterations:

Sprint 1 – Requirement Analysis and Planning

- Identify system objectives
- Gather requirements
- Analyze existing systems

Sprint 2 – System Design

- Create UML diagrams
- Design database structure
- Create user interfaces

Sprint 3 – Development

- Develop frontend using React
- Develop backend using Node.js and Express.js
- Implement database using MySQL

Sprint 4 – Integration and Testing

- Connect frontend and backend
- Test system functionalities
- Fix identified issues

Sprint 5 – Finalization

- Complete documentation
- Prepare final deliverables

Agile methodology helped the team to continuously improve the system while adapting to changes during development.

Chapter 3 - DESIGN

3.1 Design strategies

The design phase of the ShortStay system focuses on transforming the identified requirements into a structured and practical system design. The main objective of the design process is to create a system that is reliable, maintainable, scalable, and user-friendly.

The ShortStay system follows a combination of Object-Oriented Design (OOD) principles and a three-tier architecture approach. The system design separates the user interface, business logic, and data management layers to improve maintainability and future expansion.

The main design strategies used for the development of ShortStay are:

1. Modular Design

The system is divided into independent modules based on user roles and functionalities. Each module handles a specific responsibility.

The major modules include:

- User Management Module
- Property Management Module
- Booking Management Module
- Payment Management Module
- Property Verification Module
- Review and Rating Module

This approach makes the system easier to develop, test, and maintain.

2. Role-Based Design

Since ShortStay supports five different user categories, the system was designed with role-based access control.

The user roles are:

- Guest
- Host
- Admin

- Accountant
- Verifier

Each role receives access only to the functionalities relevant to their responsibilities.

3. Database-Centered Design

A structured relational database design was created to manage system information effectively.

The database stores:

- User information
- Property details
- Booking records
- Payment information
- Verification details
- Reviews

Entity Relationship (ER) modeling was used to identify relationships between system entities.

4. Responsive User Interface Design

The user interface was designed with usability as a priority. The system provides simple navigation and responsive layouts to support different types of users.

The interface design focuses on:

- Easy navigation
- Clear information presentation
- Consistent design components
- User-friendly interactions

3.2 Justification for the selected design strategy

The selected design strategies were chosen to ensure that the ShortStay system can efficiently satisfy user requirements and support future improvements.

The modular design approach allows developers to work on different system components independently. Since ShortStay contains multiple functional areas such as booking, payments, and verification, separating these modules reduces complexity and improves development efficiency.

The role-based design approach was selected because the system contains different users with different responsibilities. For example, guests should manage bookings, while verifiers should only access property inspection-related functions. This improves security and prevents unauthorized access.

The three-tier architecture approach was selected because it provides a clear separation between the frontend, backend, and database layers. This makes the system easier to maintain, test, and scale.

The object-oriented approach was selected because the system consists of real-world entities such as users, properties, bookings, and payments. Representing these entities as objects improves system organization and code reusability.

3.3 Architectural design

The ShortStay system follows a three-tier client-server architecture consisting of:

1. Presentation Layer
2. Business Logic Layer
3. Data Access Layer

Presentation Layer (Frontend)

The presentation layer is responsible for user interaction with the system.

Technology used:

- React.js

Responsibilities:

- Display user interfaces
- Handle user interactions
- Send requests to backend services
- Display system responses

Examples:

- Guest dashboard
- Host dashboard
- Admin dashboard
- Accountant dashboard

- Verifier dashboard

Business Logic Layer (Backend)

The business logic layer handles system operations and application rules.

Technology used:

- Node.js
- Express.js

Responsibilities:

- Process user requests
- Apply business rules
- Manage authentication
- Handle booking operations
- Manage verification workflows

Examples:

- Booking validation
- Property approval logic
- Payment processing logic

Data Access Layer (Database)

The data layer stores and manages system information.

Technology used:

- MySQL

Responsibilities:

- Store user details
- Store accommodation information
- Maintain booking records
- Store payment transactions
- Store verification results

The overall architecture ensures clear separation between components and improves system scalability.

3.4 Object-Oriented Methodical approach

The ShortStay system follows an Object-Oriented Design approach where system components are represented as objects with attributes and behaviors.

The main objects identified in the system are:

User

Attributes:

- User ID
- Name
- Email
- Password
- Role

Behaviors:

- Login()
- Logout()
- UpdateProfile()

Guest

Attributes:

- Guest ID

Behaviors:

- SearchProperty()
- CreateBooking()
- SubmitReview()

Host

Attributes:

- Host ID

Behaviors:

- AddProperty()
- UpdateProperty()
- ManageBooking()

Property

Attributes:

- Property ID
- Location
- Price
- Availability
- Description

Behaviors:

- UpdateDetails()
- ChangeAvailability()

Booking

Attributes:

- Booking ID
- Booking Date
- Status

Behaviors:

- ConfirmBooking()
- CancelBooking()

Payment

Attributes:

- Payment ID
- Amount
- Payment Status

Behaviors:

- ProcessPayment()

Verification

Attributes:

- Verification ID
- Status
- Inspection Date

Behaviors:

- ApproveProperty()
- RejectProperty()

3.5 High-Level Use Case Diagram for the Proposed System

The Use Case Diagram represents interactions between users and the ShortStay system.

The main actors are:

- Guest
- Host
- Admin
- Accountant
- Verifier

Main use cases:

Guest

- Register/Login
- Search Properties
- View Property Details
- Book Accommodation
- Make Payment
- Submit Review

Host

- Manage Profile
- Add Property
- Update Property Details
- Manage Bookings

Admin

- Manage Users
- Manage Properties
- Monitor System

Accountant

- Monitor Transactions
- Manage Payments
- Handle Refunds

Verifier

- Review Verification Requests
- Inspect Properties
- Approve/Reject Properties

(Insert Use Case Diagram Here)

3.6 ER Diagram for the Proposed System

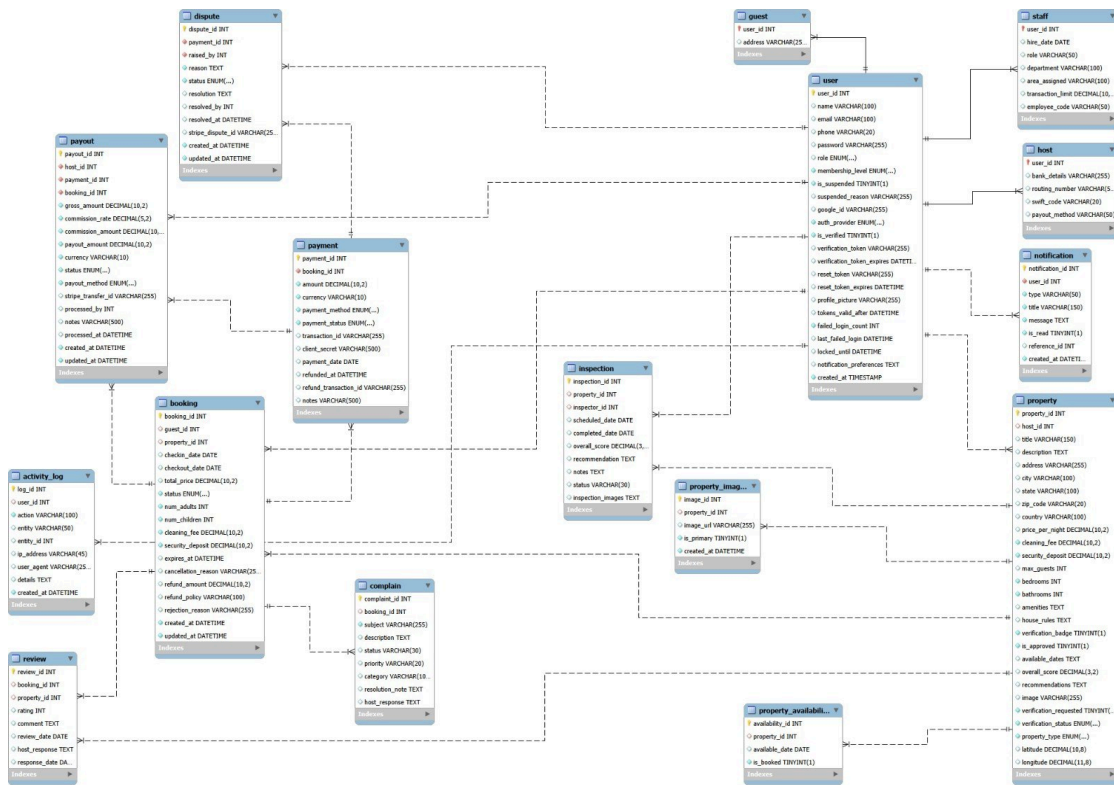
The Entity Relationship Diagram represents the database structure of the ShortStay system.

Main entities:

- User
- Host
- Guest
- Staff
- Property
- Booking
- Payment
- Review
- Inspection
- Notification
- Complain

Main relationships:

- A User can create multiple Properties.
- A Guest can make multiple Bookings.
- A Property can have multiple Bookings.
- A Booking has one Payment.
- A Property has one Inspection record.
- A Guest can submit Reviews.



3.6 Class Diagram for the Proposed System

The Class Diagram represents the object-oriented structure of the ShortStay system.

Main classes:

- User
- Guest
- Host
- Staff
- Property
- Booking
- Payment
- Complain
- Dispute
- Payout
- Activity log
- Inspection
- Notification

- Property availability

The diagram shows:

- Classes
- Attributes
- Methods
- Relationships

(Insert Class Diagram Here)

3.7 User Interface Designing for the System

The user interface of ShortStay was designed to provide a simple and efficient experience for all users.

The UI design focuses on:

- Simple navigation
- Clear information display
- Responsive layouts
- Role-based dashboards

Major interfaces include:

Guest Interface

- Login page
- Property search page
- Property details page
- Booking page

Host Interface

- Host dashboard
- Add property page

- Property management page

Admin Interface

- Admin dashboard
- User management
- Property management

Accountant Interface

- Payment dashboard
- Transaction management

Verifier Interface

- Verification dashboard
- Property inspection interface

The interface designs were created to ensure usability and consistency across the entire system.

(Insert UI screenshots/Figma designs here)

Chapter 4 – IMPLEMENTATION

4.1 Introduction

This chapter is about how we built the ShortStay website. We made it after finishing the planning and design stages. The website lets guests find and book short-term stays. It also helps hosts, administrators, payment managers and field inspectors do their jobs. We used React for the websites part, Node.js and Express for the back part and MySQL to store data. We focused on making sure people can only do what they're supposed to do on the site. We also made it easy to list properties, manage bookings, handle payments online, verify properties and send reports and notifications.

The goal was to make a platform that's both secure and easy to use. Guests can book stays, while hosts and staff can manage their work. They can do it all through dashboards made just for their roles. The back part of the site is organized into parts. There are routes, controllers, models, middleware and utility files. This makes it easy to fix and add to the site later. The ShortStay website was built to be easy to use and maintain.

4.2 Implementation Environment

The implementation environment defines the hardware, software and development setup used to build and test the ShortStay system. Since the project is a web-based system, the main development environment required a modern web browser, Node.js runtime, MySQL database server and a code editor.

Table 4.1 Development Environment Requirements

Category	Requirement / Tool	Purpose
Operating System	Windows 10/11, Linux or macOS	Used as the development platform for frontend and backend implementation
Processor	Intel Core i3 or above	Sufficient processing power for running local development servers
Memory	4 GB RAM minimum, 8 GB recommended	Needed to run the frontend, backend and database tools smoothly

Frontend Runtime	Node.js with npm	Required to install packages and run the React/Vite frontend
Backend Runtime	Node.js	Executes the Express REST API server
Database Server	MySQL Server	Stores users, properties, bookings, payments, reviews and inspection data
Database Tool	MySQL Workbench / SQL client	Used to create and manage the ShortStay database
Code Editor	Visual Studio Code	Used for writing and managing source code
API Testing Tool	Postman	Used for testing backend endpoints
Browser	Google Chrome / Microsoft Edge / Firefox	Used for frontend testing and user acceptance walkthroughs
Version Control	Git and GitHub	Used for source-code management and team collaboration

During development we run the frontend. Backend as separate services. The React frontend is served using the Vite development server. The Express backend runs as an API service. The backend talks to the MySQL database using Sequelize. We use environment variables to store configuration values. These values include database credentials, JWT secret keys, the URL for the frontend, the URL for the backend and keys for third-party APIs. The React frontend and backend services work together in this setup. The MySQL database and Sequelize help manage data. The environment variables keep our configuration safe and organized.

4.3 System Developed Tools and Technologies

The ShortStay system was implemented using modern web application technologies. The selected tools support modular development, fast interface building, secure authentication, role-based access control and relational data management.

Table 4.2 System Development Tools and Technologies

Area	Technology	Usage in ShortStay
Frontend	React.js	Developed interactive pages and role-based dashboards

Frontend Routing	React Router DOM	Handled public, guest, host, admin, payment manager and inspector routes
Frontend API Access	Axios	Sent HTTP requests from the frontend to backend REST APIs
Frontend Styling	Tailwind CSS, CSS	Created responsive layouts and consistent user interface styles
Frontend Charts	Recharts	Displayed dashboard and report-related charts
Frontend Icons	Lucide React	Used icons for navigation and dashboard actions
Build Tool	Vite	Provided fast frontend development and production builds
Backend	Node.js and Express.js	Implemented REST API endpoints and backend business logic
Database ORM	Sequelize with mysql2	Mapped JavaScript models to MySQL database tables
Database	MySQL	Stored structured application data
Authentication	JWT and bcryptjs	Implemented secure login, password hashing and token-based sessions
Validation and Security	express-validator, express-rate-limit, CORS	Validated inputs, controlled access and reduced abuse attempts
File Uploading	Multer	Handled property, inspection and profile image uploads
Payments	Stripe	Supported secure online payment workflow
Email	Nodemailer / email templates	Sent verification, reset, booking and payment emails
Real-time Updates	Socket.IO	Supported real-time notification functionality
API Documentation	Swagger UI	Displayed backend API documentation for developers

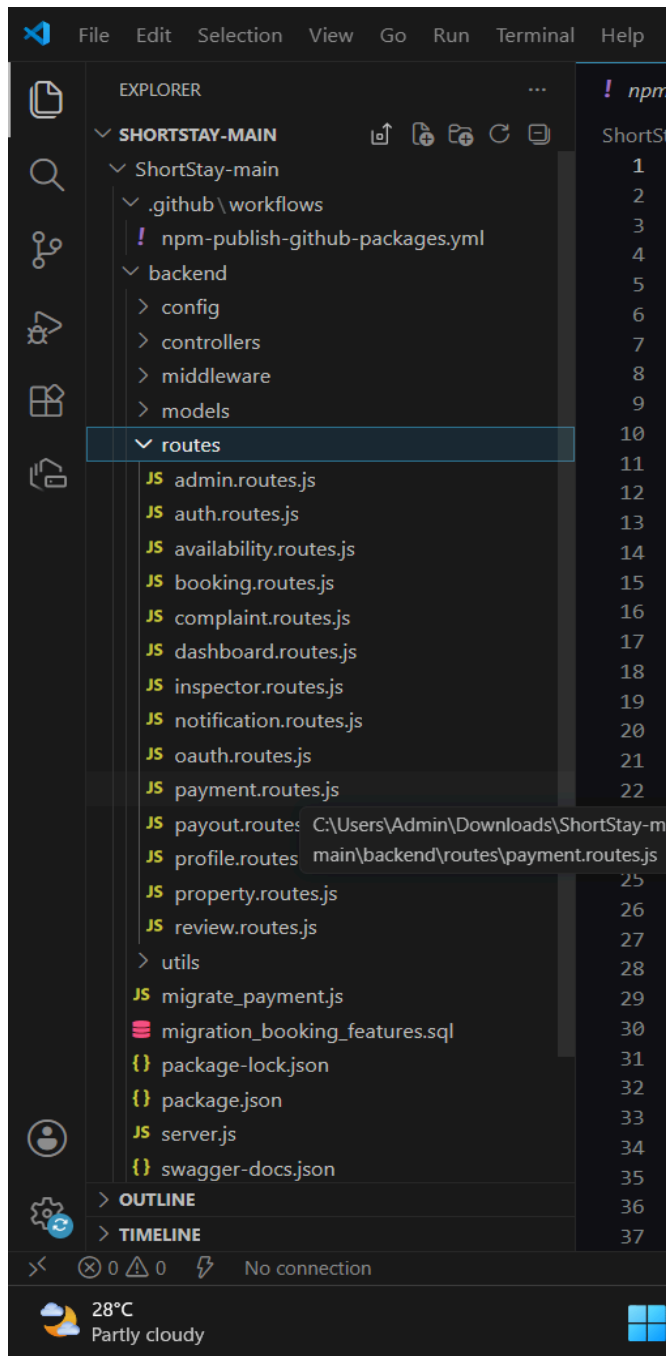


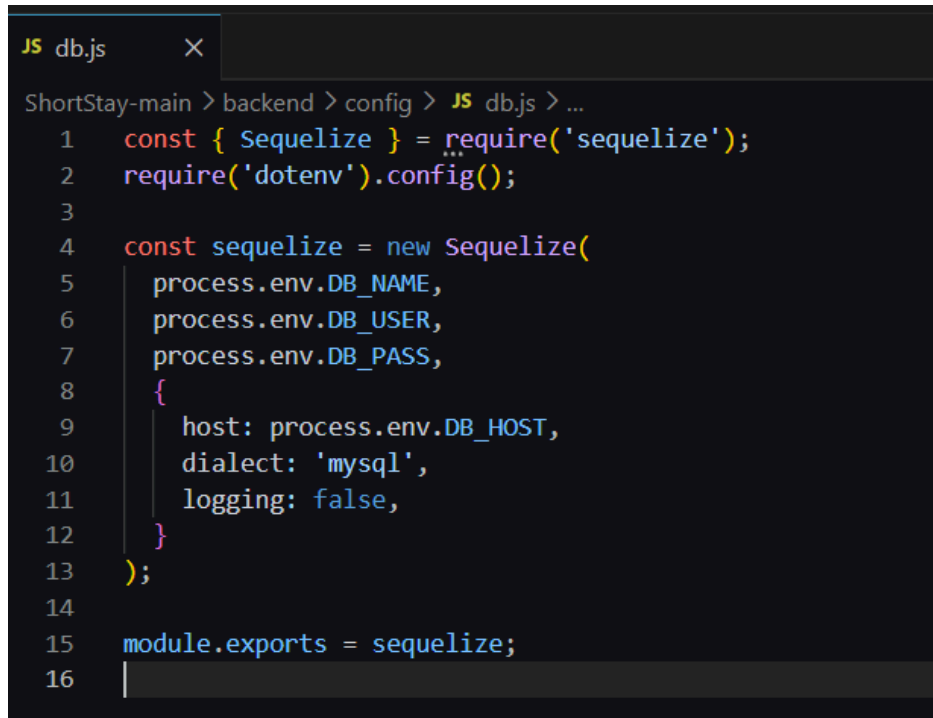
Figure 4.1 System module folder structure of ShortStay

4.4 Major Code Segments

This section explains the most important code segments used in the system implementation. The code screenshots are selected from the actual project files because they show the main implementation logic of the ShortStay application.

4.4.1 Database Connection

The backend uses Sequelize to connect the Node.js application with the MySQL database. Database credentials are not hard-coded in the application. Instead, values such as database name, database user, password and host are loaded through environment variables. This approach improves security and allows the same project to be configured for different environments

A screenshot of a code editor window titled 'db.js'. The editor shows the following JavaScript code:

```
1  const { Sequelize } = require('sequelize');
2  require('dotenv').config();
3
4  const sequelize = new Sequelize(
5    process.env.DB_NAME,
6    process.env.DB_USER,
7    process.env.DB_PASS,
8    {
9      host: process.env.DB_HOST,
10     dialect: 'mysql',
11     logging: false,
12   }
13 );
14
15 module.exports = sequelize;
16
```

Figure 4.2 Code segment for database connection

4.4.2 Server Initialization and API Route Registration

The Express server is the main entry point of the backend. It configures middleware such as request logging, response-time monitoring, rate limiting, CORS, JSON parsing and Passport initialization. After middleware configuration, each system module is registered as a separate API route. This keeps the backend modular and makes the system easier to maintain.

```

JS db.js JS server.js X
ShortStay-main > backend > JS server.js > ...
21
22 // — Non-functional middleware —————
23 app.use(requestLogger); // HTTP request log (morgan)
24 app.use(responseTimeMonitor); // X-Response-Time header + slow-req warning
25 app.use(globalRateLimit); // 100 req / 15 min / IP
26
27 app.use(cors({
28   origin: process.env.FRONTEND_URL,
29   credentials: true
30 }));
31
32 // — Stripe webhook — must receive RAW body before express.json() parses it —
33 app.use(
34   '/api/payments/webhook',
35   express.raw({ type: 'application/json' })
36 );
37
38 app.use(express.json());
39 app.use(passport.initialize());
40 app.use('/uploads', express.static('uploads'));
41 app.use('/uploads/profiles', express.static('uploads/profiles'));
42
43 // — API Routes —————
44 app.use('/api/auth', authRateLimit, require('./routes/auth.routes'));
45 app.use('/api/auth', authRateLimit, require('./routes/oauth.routes'));
46 app.use('/api/properties', require('./routes/property.routes'));
47 app.use('/api/bookings', require('./routes/booking.routes'));
48 app.use('/api/payments', require('./routes/payment.routes'));
49 app.use('/api/payouts', require('./routes/payout.routes'));
50 app.use('/api/reviews', require('./routes/review.routes'));
51 app.use('/api/complaints', require('./routes/complaint.routes'));
52 app.use('/api/admin', require('./routes/admin.routes'));
53 app.use('/api/inspector', require('./routes/inspector.routes'));
54 app.use('/api/availability', require('./routes/availability.routes'));
55 app.use('/api/profile', require('./routes/profile.routes'));
56 app.use('/api/notifications', require('./routes/notification.routes'));
57 app.use('/api/dashboard', require('./routes/dashboard.routes'));

```

Figure 4.3 Code segment for Express server and route registration

4.4.3 Login, Account Checks and JWT Generation

The authentication controller handles user login. It checks whether the user exists, whether the account is suspended, whether email verification is required, and whether the entered password matches the hashed password stored in the database. The system also includes a brute-force protection mechanism that locks the account temporarily after repeated failed login attempts. When the login is successful, a JWT token is generated and returned to the frontend.


```

ShortStay-main > backend > controllers > JS auth.controller.js > ...
224 exports.login = async (req, res) => {
225   try {
226     const { email, password, remember_me } = req.body;
227
228     const user = await User.findOne({ where: { email } });
229     if (!user) {
230       return res.status(404).json({ message: 'User not found' });
231     }
232
233     // — DB-persisted brute-force check —
234     if (user.locked_until && new Date() < new Date(user.locked_until)) {
235       const remainingTime = Math.ceil(
236         (new Date(user.locked_until) - Date.now()) / 60000
237       );
238       return res.status(429).json({
239         message: `Too many failed login attempts. Try again in ${remainingTime} minutes
240       });
241     }
242     // Auto-reset when lock period has passed
243     if (user.locked_until && new Date() >= new Date(user.locked_until)) {
244       await user.update({ failed_login_count: 0, last_failed_login: null, locked_until:
245     }
246
247     // Check if suspended
248     if (user.is_suspended) {
249       return res.status(403).json({
250         message: 'Your account has been suspended.',
251         reason: user.suspended_reason || 'Please contact support.'
252       });
253     }
254
255     // Block Google-only accounts (no password set) from password login
256     if (!user.password) {
257       return res.status(400).json({
258         message: 'This account uses Google login. Please sign in with Google.',
259       });
260     }

```

```

ShortStay-main > backend > controllers > JS auth.controller.js > ...
224 exports.login = async (req, res) => {
225   try {
226     const { email, password, remember_me } = req.body;
227
228     const user = await User.findOne({ where: { email } });
229     if (!user) {
230       return res.status(404).json({ message: 'User not found' });
231     }
232
233     // — DB-persisted brute-force check —
234     if (user.locked_until && new Date() < new Date(user.locked_until)) {
235       const remainingTime = Math.ceil(
236         (new Date(user.locked_until) - Date.now()) / 60000
237       );
238       return res.status(429).json({
239         message: `Too many failed login attempts. Try again in ${remainingTime} minutes
240       });
241     }
242     // Auto-reset when lock period has passed
243     if (user.locked_until && new Date() >= new Date(user.locked_until)) {
244       await user.update({ failed_login_count: 0, last_failed_login: null, locked_until:
245     }
246
247     // Check if suspended
248     if (user.is_suspended) {
249       return res.status(403).json({
250         message: 'Your account has been suspended.',
251         reason: user.suspended_reason || 'Please contact support.'
252       });
253     }
254
255     // Block Google-only accounts (no password set) from password login
256     if (!user.password) {
257       return res.status(400).json({
258         message: 'This account uses Google login. Please sign in with Google.',
259       });
260     }

```

Figure 4.4 Code segment for login and JWT generation

4.4.4 JWT Authentication Middleware

Protected backend routes use JWT middleware to ensure that only authenticated users can access restricted API endpoints. The middleware checks for a Bearer token in the request header, verifies the token using the JWT secret and loads the related user record. It also checks whether the account is suspended or whether the token has been invalidated after logout or password change.

```
ShortStay-main > backend > middleware > JS auth.middleware.js > authMiddleware
1  const jwt = require('jsonwebtoken');
2  require('dotenv').config();
3
4  const authMiddleware = async (req, res, next) => {
5    const authHeader = req.headers.authorization;
6
7    if (!authHeader || !authHeader.startsWith('Bearer ')) {
8      return res.status(401).json({ message: 'No token provided' });
9    }
10
11    const token = authHeader.split(' ')[1];
12
13    try {
14      const decoded = jwt.verify(token, process.env.JWT_SECRET);
15
16      const { User } = require('../models/index');
17      const user = await User.findPk(decoded.user_id);
18
19      if (!user) {
20        return res.status(404).json({ message: 'User not found' });
21      }
22
23      // Check if token was issued before the invalidation timestamp
24      // (covers logout, password change, and any admin-forced invalidation)
25      if (user.tokens_valid_after) {
26        const tokenIssuedAt = new Date(decoded.iat * 1000);
27        if (tokenIssuedAt < new Date(user.tokens_valid_after)) {
28          return res.status(401).json({
29            message: 'Token has been invalidated. Please login again.',
30          });
31        }
32      }
33
34      if (user.is_suspended) {
35        return res.status(403).json({
36          message: 'Your account has been suspended.',
37          reason: user.suspended_reason || 'Please contact support.',
38        });
39      }
40
41      next();
42    } catch (error) {
43      return res.status(401).json({ message: 'Invalid token' });
44    }
45  };
```

Figure 4.5 Code segment for JWT authentication middleware

4.4.5 Frontend Role-Based Route Protection

The frontend also contains role-based route protection. The ProtectedRoute component checks the currently logged-in user and allows access only to pages that match the user role. For example, a guest can access guest pages, while the admin dashboard is restricted to the admin role. This improves the user experience and prevents unauthorized users from viewing role-specific screens.

```

53 function ProtectedRoute({ children, role }) {
54   const { user } = useAuth();
55   // In mock mode – skip all auth guards so every page is accessible
56   if (MOCK) return children;
57   if (!user) return <Navigate to="/login" replace />;
58   if (role && user.role !== role) return <Navigate to="/" replace />;
59   return children;
60 }
61
62 function RoleRedirect() {
63   const { user } = useAuth();
64   if (!user) return <Navigate to="/" replace />;
65   const home = { guest: '/guest/browse', host: '/host/listings', admin: '/admin/dashboard', f
66   return <Navigate to={home[user.role] || '/'} replace />;
67 }

```

```

90  /* Guest */
91  <Route path="/guest" element={<ProtectedRoute role="guest"><GuestDashboard /></ProtectedRoute>} />
92  <Route path="/guest/browse" element={<ProtectedRoute role="guest"><BrowseListings /></ProtectedRoute>} />
93  <Route path="/guest/property/:id" element={<ProtectedRoute role="guest"><GuestPropertyDetail /></ProtectedRoute>} />
94  <Route path="/guest/bookings" element={<ProtectedRoute role="guest"><MyBookings /></ProtectedRoute>} />
95  <Route path="/guest/pay/:bookingId" element={<ProtectedRoute role="guest"><PaymentPage /></ProtectedRoute>} />
96  <Route path="/guest/wallet" element={<ProtectedRoute role="guest"><GuestWallet /></ProtectedRoute>} />
97  <Route path="/guest/reviews" element={<ProtectedRoute role="guest"><GuestReviews /></ProtectedRoute>} />
98  <Route path="/guest/settings" element={<ProtectedRoute role="guest"><GuestSettings /></ProtectedRoute>} />
99
100 /* Host */
101 <Route path="/host" element={<ProtectedRoute role="host"><HostDashboard /></ProtectedRoute>} />
102 <Route path="/host/listings" element={<ProtectedRoute role="host"><HostListings /></ProtectedRoute>} />
103 <Route path="/host/listings/new" element={<ProtectedRoute role="host"><PropertyForm /></ProtectedRoute>} />
104 <Route path="/host/listings/edit/:id" element={<ProtectedRoute role="host"><PropertyForm /></ProtectedRoute>} />
105 <Route path="/host/bookings" element={<ProtectedRoute role="host"><HostBookings /></ProtectedRoute>} />
106 <Route path="/host/earnings" element={<ProtectedRoute role="host"><HostDashboard /></ProtectedRoute>} />
107 <Route path="/host/reviews" element={<ProtectedRoute role="host"><HostReviews /></ProtectedRoute>} />
108 <Route path="/host/settings" element={<ProtectedRoute role="host"><GuestSettings /></ProtectedRoute>} />
109
110 /* Admin */
111 <Route path="/admin/dashboard" element={<ProtectedRoute role="admin"><AdminDashboard /></ProtectedRoute>} />
112 <Route path="/admin/users" element={<ProtectedRoute role="admin"><AdminUsers /></ProtectedRoute>} />
113 <Route path="/admin/properties" element={<ProtectedRoute role="admin"><AdminProperties /></ProtectedRoute>} />
114 <Route path="/admin/bookings" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
115 <Route path="/admin/payments" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
116 <Route path="/admin/reports" element={<ProtectedRoute role="admin"><AdminReports /></ProtectedRoute>} />
117 <Route path="/admin/verifier" element={<ProtectedRoute role="admin"><AdminProperties /></ProtectedRoute>} />
118 <Route path="/admin/accountant" element={<ProtectedRoute role="admin"><AdminPayments /></ProtectedRoute>} />
119 <Route path="/admin/host" element={<ProtectedRoute role="admin"><AdminUsers /></ProtectedRoute>} />
120 <Route path="/admin/settings" element={<ProtectedRoute role="admin"><AdminDashboard /></ProtectedRoute>} />
121
122 /* Inspector */
123 <Route path="/inspector/inspections" element={<ProtectedRoute role="field_inspector"><InspectorDashboard /></ProtectedRoute>} />
124 <Route path="/inspector/settings" element={<ProtectedRoute role="field_inspector"><InspectorDashboard /></ProtectedRoute>} />

```

Figure 4.6 Code segment for frontend protected routes

4.4.6 Frontend API Communication

The frontend uses a reusable Axios instance to communicate with the backend REST API. The API helper automatically attaches the JWT token from local storage to request headers. It also handles unauthorized responses by clearing the local user session and redirecting the user to the login page. In mock mode, the same helper can return sample data without connecting to the backend, which is useful for UI demonstration and frontend testing.

```

ShortStay-main > frontend > src > JS api.js > ...
1  import axios from 'axios';
2  import { mockRequest } from './mockApi';
3
4  const MOCK = import.meta.env.VITE_MOCK_MODE === 'true';
5
6  const api = axios.create({ baseURL: 'http://localhost:5000/api' });
7
8  api.interceptors.request.use(cfg => {
9    const token = localStorage.getItem('token');
10    if (token) cfg.headers.Authorization = `Bearer ${token}`;
11    return cfg;
12  });
13
14  api.interceptors.response.use(
15    res => res,
16    err => {
17      if (err.response?.status === 401) {
18        localStorage.removeItem('token');
19        localStorage.removeItem('user');
20        window.location.href = '/login';
21      }
22      return Promise.reject(err);
23    }
24  );
25

```

Figure 4.7 Code segment for Axios API helper

4.4.7 Booking Creation and Availability Validation

The booking controller validates property availability before creating a booking. It checks whether the selected property is approved, validates check-in and checkout dates, calculates the number of nights and checks whether all required dates are available. This logic prevents invalid bookings and reduces double-booking issues.

```

ShortStay-main > backend > controllers > JS booking.controller.js > ...
68 exports.makeBooking = async (req, res) => {
69   try {
70     const { property_id, checkin_date, checkout_date } = req.body;
71
72     // 1. Validate property
73     const property = await Property.findOne({
74       where: { property_id, is_approved: true },
75     });
76     if (!property) {
77       return res.status(404).json({ message: 'Property not found or not approved' });
78     }
79
80     // 2. Validate dates
81     const checkin = new Date(checkin_date);
82     const checkout = new Date(checkout_date);
83     const nights = Math.ceil((checkout - checkin) / (1000 * 60 * 60 * 24));
84     if (nights <= 0) {
85       return res.status(400).json({ message: 'Invalid dates: checkout must be after checkin' });
86     }
87     if (checkin < new Date()) {
88       return res.status(400).json({ message: 'Check-in date cannot be in the past' });
89     }
90
91     // 3. Check availability BEFORE creating booking
92     const requiredDates = getDateRange(checkin_date, checkout_date);
93     const availableDates = await PropertyAvailability.findAll({
94       where: {
95         property_id,
96         available_date: { [Op.in]: requiredDates },
97         is_booked: false,
98       },
99     });
100     if (availableDates.length !== requiredDates.length) {
101       return res.status(409).json({
102         message: 'Property is not available for the selected dates',
103         required_dates: requiredDates.length,
104         available_dates: availableDates.length,

```

```

105   });
106 }
107
108 // 4. Prevent double booking – check for overlapping PENDING/CONFIRMED bookings
109 const overlap = await Booking.findOne({
110   where: {
111     property_id,
112     status: { [Op.in]: ['pending', 'confirmed'] },
113     [Op.or]: [
114       { checkin_date: { [Op.between]: [checkin_date, checkout_date] } },
115       { checkout_date: { [Op.between]: [checkin_date, checkout_date] } },
116       {
117         checkin_date: { [Op.lte]: checkin_date },
118         checkout_date: { [Op.gte]: checkout_date },
119       },
120     ],
121   },
122 });
123 if (overlap) {
124   return res.status(409).json({
125     message: 'This property already has an active booking overlapping your selected dates',
126   });
127 }
128
129 // 5. Create booking with 24-hour expiry window
130 const total_price = nights * property.price_per_night;
131 const expires_at = new Date(Date.now() + 24 * 60 * 60 * 1000); // +24 h
132
133 const booking = await Booking.create({
134   guest_id: req.user.user_id,
135   property_id,
136   checkin_date,
137   checkout_date,
138   total_price,
139   status: 'pending',
140   expires_at,

```

Figure 4.8 Code segment for booking creation and availability validation

4.4.8 Payment Intent Creation

The payment module supports online payment processing. The Stripe PaymentIntent flow creates a pending payment record and returns a client secret to the frontend. This allows the frontend to complete card confirmation securely while the backend stores the payment record and links it to the booking.

```
ShortStay-main > backend > controllers > JS payment.controller.js > createPaymentIntent > createPaymentIntent
83  exports.createPaymentIntent = async (req, res) => {
84    try {
85      const stripe = getStripe();
86      const { booking_id } = req.body;
87
88      const booking = await Booking.findByPk(booking_id, {
89        include: [{ model: Property, as: 'property', attributes: ['title', 'address'] }],
90      });
91      if (!booking) return res.status(404).json({ message: 'Booking not found' });
92      if (booking.guest_id !== req.user.user_id)
93        return res.status(403).json({ message: 'Not authorized' });
94      if (booking.status !== 'confirmed')
95        return res.status(400).json({ message: 'Payment can only be made for confirmed bookings' });
96
97      const existing = await Payment.findOne({
98        where: { booking_id, payment_status: 'completed' },
99      });
100     if (existing) return res.status(400).json({ message: 'Booking already paid' });
101
102     // Amount in cents (Stripe expects smallest currency unit)
103     const amountCents = Math.round(parseFloat(booking.total_price) * 100);
104
105     const intent = await stripe.paymentIntents.create({
106       amount: amountCents,
107       currency: 'usd',
108       metadata: {
109         booking_id: String(booking_id),
110         guest_id: String(req.user.user_id),
111       },
112       description: `ShortStay Booking #${booking_id} - ${booking.property?.title}`,
113     });
114
115     // Create a pending payment record
116     const payment = await Payment.create({
117       booking_id,
118       amount: booking.total_price,
119       currency: 'USD',
120
121       currency: 'USD',
122       payment_method: 'stripe',
123       payment_status: 'pending',
124       transaction_id: intent.id,
125       client_secret: intent.client_secret,
126       payment_date: new Date(),
127     });
128
129     res.status(201).json({
130       message: 'Payment intent created',
131       client_secret: intent.client_secret,
132       payment_id: payment.payment_id,
133       amount: booking.total_price,
134       currency: 'USD',
135     });
136   } catch (err) {
```

Figure 4.9 Code segment for Stripe PaymentIntent creation

4.5 System module structure

The ShortStay project is divided into frontend, backend and database layers. This separation makes the application easier to understand, test and extend. The frontend manages the user interface and user interaction, the backend handles business logic and API endpoints, and the database stores persistent records.

Table 4.3 System Module Structure

Layer / Module	Main Files or Folders	Description
Frontend public pages	frontend/src/pages/public	Home, About, Contact, property detail and access portal pages
Frontend authentication pages	frontend/src/pages/auth	Login, registration and forgot-password interfaces
Guest module	frontend/src/pages/guest	Browse listings, view property details, make payments, view bookings, wallet, reviews and settings
Host module	frontend/src/pages/host	Host dashboard, listing management, property form, bookings and reviews
Admin module	frontend/src/pages/admin	Admin dashboard, user management, property approval, complaints, payments and reports
Payment manager module	frontend/src/pages/payment_manager	Payment dashboard and payment reports
Field inspector module	frontend/src/pages/inspector	Inspection dashboard and property verification workflow
Reusable UI components	frontend/src/components	Dashboard layout, sidebar, top bar, public navigation, footer and role switcher
Backend configuration	backend/config	Database, email, passport and Swagger configuration
Backend controllers	backend/controllers	Business logic for auth, booking, property, payment, reports, inspection and other modules
Backend routes	backend/routes	REST API route definitions for each module
Backend middleware	backend/middleware	Authentication, authorization, validators, upload handlers and system middleware

Backend models	backend/models	Sequelize models and relationships for database tables
Backend utilities	backend/utills	Activity logger, notification sender, email templates, membership logic, cron job and websocket utilities
Database script	Files/shortstay.sql	SQL structure for user, property, booking, payment, review, inspection and related tables

The backend follows a controller-based structure. Each route file forwards requests to the relevant controller, and controllers use Sequelize models to read from or write to the database. Middleware files are used for repeated cross-cutting tasks such as authentication, authorization, file uploading and input validation.

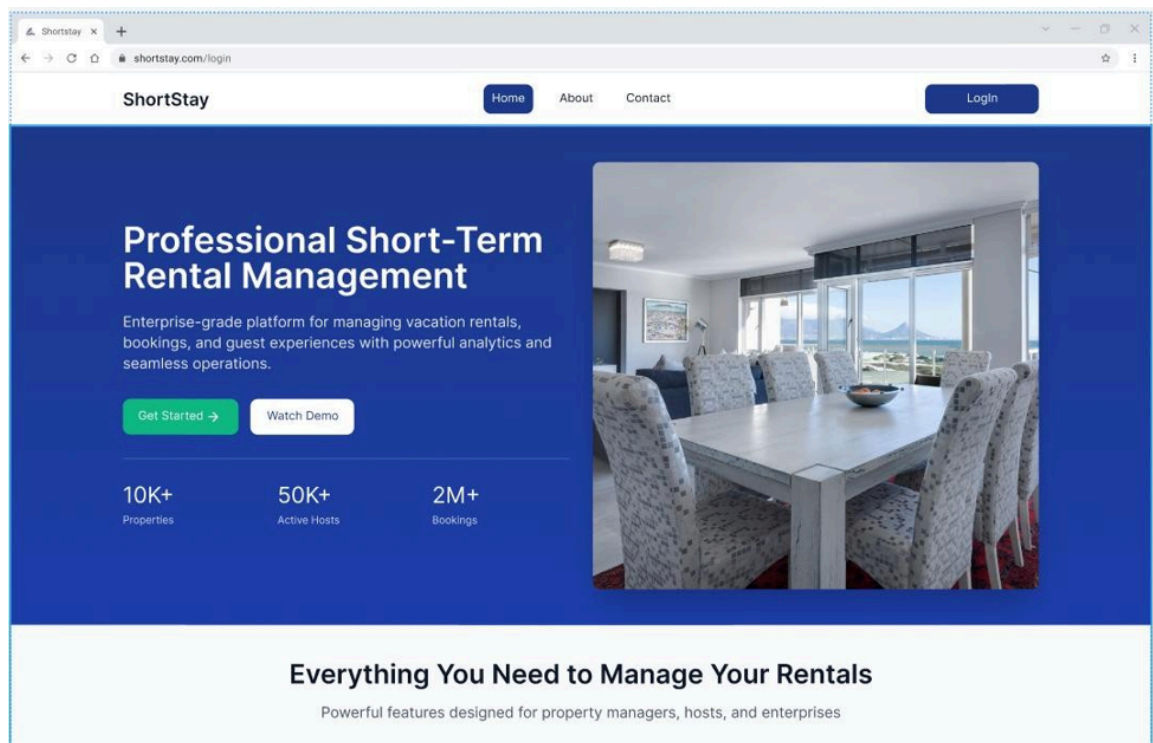


Figure 4.10 Home page interface

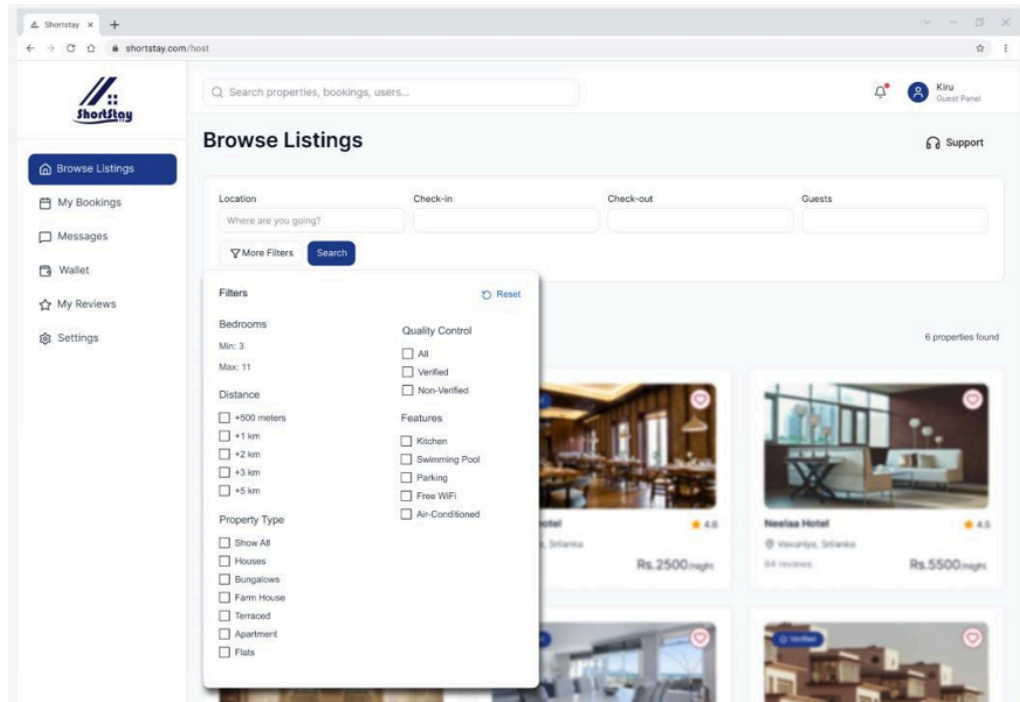


Figure 4.11 Property search and filtering interface

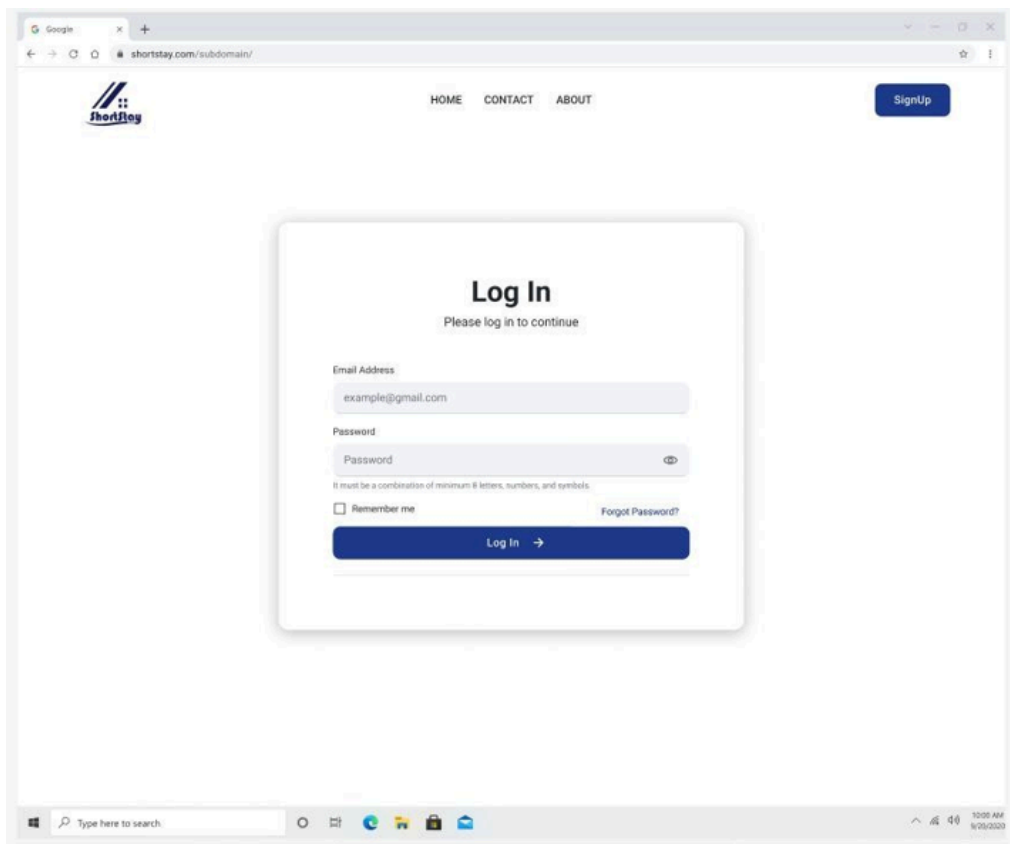


Figure 4.12 Login page interface

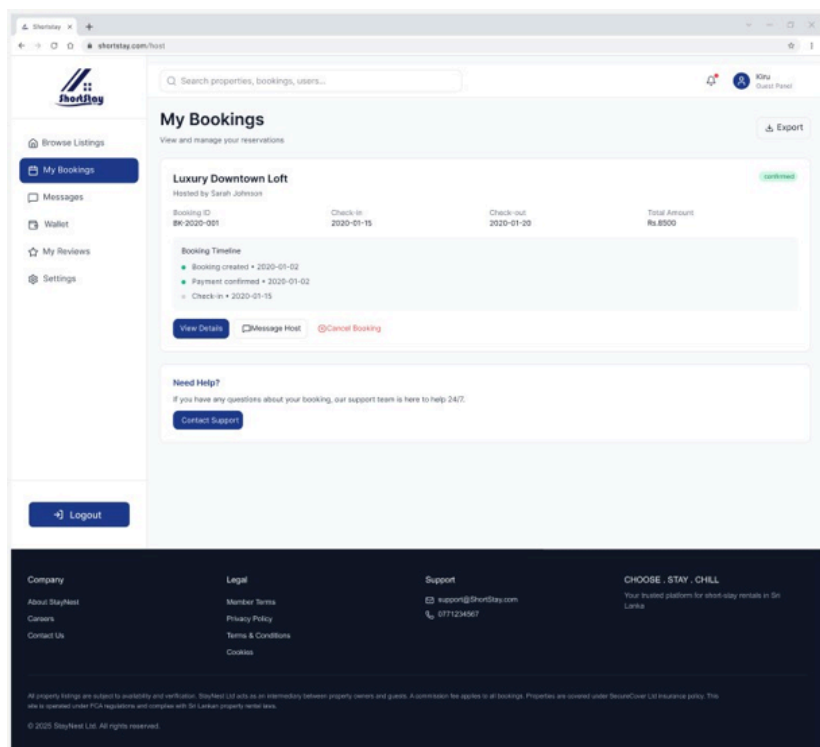


Figure 4.13 Guest booking history interface

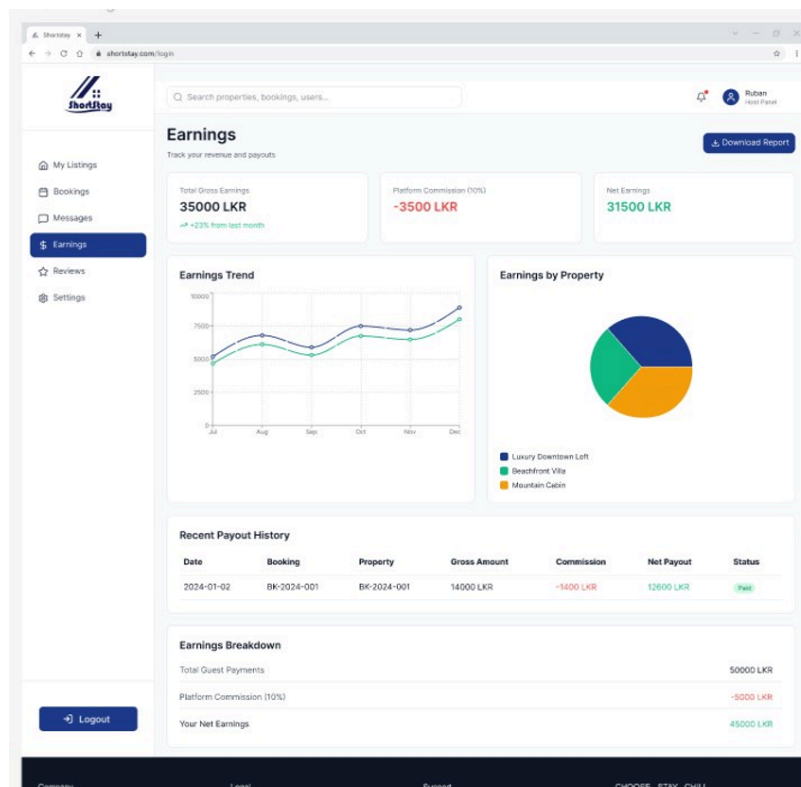


Figure 4.14 Host earnings/dashboard interface

4.6 Re-usable Components

Reusable components and utilities were used to reduce code duplication and improve consistency across the system. These reusable components support maintainability because a common layout, navigation bar, sidebar, authentication state and API handler can be changed in one place instead of updating every page separately.

Table 4.4 Reusable Components and Utilities

Reusable Component / Utility	Location	Reusability Purpose
DashboardLayout	frontend/src/components/DashboardLayout.jsx	Provides a consistent dashboard layout for role-based panels
Sidebar	frontend/src/components/Sidebar.jsx	Displays role-specific side navigation links
TopBar	frontend/src/components/TopBar.jsx	Shows the page header, search field and user information
PublicNav	frontend/src/components/PublicNav.jsx	Common navigation bar for public pages
Footer	frontend/src/components/Footer.jsx	Common footer for the public and dashboard pages
AuthContext	frontend/src/AuthContext.jsx	Stores login state and user role across the frontend
API helper	frontend/src/api.js	Central place for backend API calls and token attachment
Auth middleware	backend/middleware/auth.middleware.js	Reusable JWT validation for protected backend APIs
Role middleware	backend/middleware/role.middleware.js	Reusable role permission checks for backend APIs
Validation middleware	backend/middleware/validators.js and validate.middleware.js	Reusable input validation rules
Upload middleware	backend/middleware/upload*.js	Reusable file upload handling for properties, profiles and inspections
Activity logger	backend/utls/activityLogger.js	Records important user/system activities
Email templates	backend/utls/emailTemplates.js	Reusable email layouts for verification, booking and payment workflows
Notification utility	backend/utls/notify.js	Reusable in-app notification creation
Booking expiry cron	backend/utls/bookingExpiryCron.js	Automatically handles expired pending bookings

Websocket utility	backend/utls/websocket.js	Reusable real-time notification setup
-------------------	---------------------------	---------------------------------------

Using reusable components helped the project maintain a consistent user interface across all roles. It also improved development speed because common functions such as authentication, authorization, notifications and API handling did not need to be rewritten for every module.

Chapter 5 – EVALUATION

5.1 Introduction

This chapter evaluates the ShortStay system by describing the testing approach, testing techniques, test plan, test cases and user acceptance testing. Evaluation is important because it confirms whether the developed system satisfies the functional and non-functional requirements identified in the earlier chapters. The system was evaluated across major modules such as authentication, property management, booking management, payment management, role-based access, reports, reviews, complaints and property inspection.

The main objective of system evaluation was to ensure that the application works correctly for each user role and that data is processed accurately between the frontend, backend and database. Testing also focused on security-related areas such as protected routes, JWT validation, user role restrictions, password hashing, account suspension and rate limiting.

5.2 System Testing

System testing was carried out to validate the overall behaviour of the ShortStay application. The frontend was checked using the browser, while backend API endpoints were tested using Postman and direct route verification. Database-related functions were verified by checking whether records such as users, properties, bookings and payments were created, updated and retrieved correctly.

The system was evaluated module by module and then as a complete end-to-end flow. For example, a host can create a property listing, the property can be approved, a guest can search for the property, make a booking, complete the payment, and later provide a review. This end-to-end flow confirms the interaction between multiple modules.

5.3 Testing Techniques

The following testing techniques were used or prepared for evaluating the ShortStay application.

Table 5.1 Testing Techniques

Testing Technique	Description	Application in ShortStay
-------------------	-------------	--------------------------

Black-box testing	Tests system behaviour without checking internal code	Used for login, registration, search, booking, payment and dashboard workflows
White-box testing	Tests internal logic and code paths	Used for authentication logic, route protection, booking validation and payment processing code
Input validation testing	Checks whether invalid inputs are rejected properly	Used for email, password, dates, property details, payment and review forms
Security testing	Checks authentication and access control	Used for JWT token validation, role-based access, account suspension and invalid token handling
Usability testing	Checks ease of use and interface clarity	Used for public pages, guest dashboard, host dashboard and admin screens
Regression testing	Retests existing features after changes	Used after bug fixing or module updates to confirm previous functions still work

5.4 Testing Methodologies

Testing was organized using different testing levels. This helped identify errors at the smallest module level and also at the full system level.

Table 5.2 Testing Methodologies

Testing Methodology	Purpose	Example in ShortStay
Unit testing	Tests individual functions or modules separately	Testing password hashing, JWT generation, date range generation and refund calculation logic
Integration testing	Tests whether connected modules work together	Testing frontend login request with backend authentication and database user records
System testing	Tests the full system as one application	Testing complete guest booking and payment workflow
User acceptance testing	Checks whether the system meets user expectations	Guest, host and admin users perform realistic tasks and provide feedback
Performance observation	Checks response and usability under normal use	Checking page loading, API response behaviour and dashboard data loading

Compatibility testing	Checks the system in different browsers	Testing screens in Chrome, Edge or Firefox
-----------------------	---	--

5.5 System Test Plan

The system test plan defines what should be tested, which tools should be used and what result is expected. The test plan covers the major modules that are important for the successful operation of the ShortStay system.

Table 5.3 System Test Plan

Test Area	Modules Covered	Testing Method	Expected Outcome
Authentication and user management	Registration, login, logout, forgot password, profile, staff creation	Black-box, white-box, security testing	Only valid users can access the system and roles are assigned correctly
Role-based access control	Guest, host, admin, payment manager and field inspector routes	Security and integration testing	Users can access only the pages and APIs permitted for their role
Property management	Create, update, delete, upload images, request verification, approve/reject listing	Functional and integration testing	Property data is stored correctly and displayed to users
Search and filtering	BrowseListings and property detail pages	Black-box and usability testing	Guests can filter and view properties accurately
Booking management	Make booking, approve, reject, cancel, booking history	Functional, integration and boundary testing	Bookings are created only for valid available dates
Payment management	Stripe intent, manual payment, refund, receipt, payout and reports	Integration and security testing	Payments are linked to bookings and status is updated correctly
Inspection and verification	Inspector dashboard, submit inspection, approve badge	Functional and workflow testing	Property verification status is updated based on inspection results
Reviews and complaints	Create review, edit review, host response, complaint status	Functional testing	Users can submit feedback and admins can manage complaints

Reports and dashboards	Admin reports, payment manager reports, dashboards	System and UI testing	Dashboard totals and report results are displayed correctly
Non-functional requirements	Security, usability, maintainability and responsiveness	Usability, compatibility and performance observation	System remains user-friendly, secure and maintainable

5.6 System Test Cases

The following test cases were prepared to validate the main features of the ShortStay system. These cases can be used during final testing and project demonstration.

Table 5.4 Test Cases for Authentication

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	User registration with valid guest details	Enter name, email, phone, password and guest role	Guest account is created and verification process starts	Pass
TC-02	User login with valid credentials	Enter verified email and correct password	System returns JWT token and redirects user to correct dashboard	Pass
TC-03	User login with invalid password	Enter valid email and wrong password	System rejects login and returns invalid password message	Pass
TC-04	Repeated invalid login attempts	Enter wrong password more than allowed attempts	System temporarily locks the account	Pass
TC-05	Logout from the system	Click logout button	Token is invalidated and user is redirected from protected pages	Pass
TC-06	Access protected route without login	Open protected route without JWT token	System redirects user or returns unauthorized response	Pass

Table 5.5 Test Cases for Property and Search Management

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-07	Host creates a property listing	Enter title, description, address, price, guests and property type	Property is saved with pending or unapproved status	Pass
TC-08	Host uploads property images	Upload valid image file	Image is stored and linked with the property	Pass
TC-09	Admin approves property	Admin selects approve action	Property becomes visible to guests as approved	Pass
TC-10	Guest searches properties	Search by location, price or filters	Matching properties are displayed	Pass
TC-11	Guest opens property detail page	Click a property card	Property details, price, amenities and reviews are displayed	Pass

Table 5.6 Test Cases for Booking Management

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-12	Guest creates booking for available dates	Select valid check-in and checkout dates	Booking is created with pending status and total price calculated	Pass
TC-13	Guest selects checkout before check-in	Select invalid date range	System rejects booking with invalid date message	Pass
TC-14	Guest books unavailable dates	Select already booked dates	System rejects booking and prevents double booking	Pass
TC-15	Host approves booking request	Host selects approve booking	Booking status changes to confirmed and notification/email is sent	Pass
TC-16	Guest cancels booking	Guest selects cancel option	Booking is cancelled and refund policy can be applied	Pass

Table 5.7 Test Cases for Payment, Reports and Feedback

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-17	Create Stripe payment intent	Guest proceeds to pay for confirmed booking	Backend creates pending payment and returns client secret	Pass
TC-18	Confirm completed payment	Payment confirmation is received	Payment status changes to completed and receipt can be viewed	Pass
TC-19	Payment manager views payment records	Open payment dashboard	Transaction list and totals are displayed	Pass
TC-20	Admin generates report	Select report page	System displays booking, user, property or revenue-related report data	Pass
TC-21	Guest submits review	Enter rating and comment for completed booking	Review is saved and displayed under property reviews	Pass
TC-22	User submits complaint	Enter complaint details for booking/property issue	Complaint is recorded and admin can update status	Pass

Table 5.8 Test Cases for Inspection, Admin and Security

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-23	Field inspector views pending inspections	Open inspector dashboard	Pending property verification tasks are displayed	Pass
TC-24	Inspector submits inspection result	Enter score, recommendation and status	Inspection record is saved and property status is updated	Pass
TC-25	Admin suspends user	Admin selects suspend and enters reason	User account becomes suspended and login is blocked	Pass
TC-26	Unauthorized role opens admin page	Guest tries to open admin route	Access is blocked by frontend/backend role control	Pass

5.7 User acceptance test

User Acceptance Testing (UAT) verifies whether the completed system meets the expectations of the intended users. In the ShortStay system, UAT should involve representatives from each user role: guest, host, admin, payment manager and field inspector. Each user should be asked to perform realistic tasks and comment on ease of use, clarity, speed and completeness of the system.

Table 5.9 User Acceptance Test Summary

User Role	UAT Task	Acceptance Criteria	Result
Guest	Register, search property, view details, create booking and complete payment	Guest can complete the booking process without staff support	Accepted
Host	Create listing, update details, view bookings and check earnings	Host can manage listings and booking information clearly	Accepted
Admin	Manage users, approve properties, view complaints and reports	Admin can monitor and control system operations	Accepted
Payment Manager	View transactions, review payment status and generate payment report	Payment records can be tracked and summarized accurately	Accepted
Field Inspector	View assigned inspections and submit inspection result	Inspector can verify property information and submit recommendations	Accepted

The UAT results indicate that the major workflows of the ShortStay system are understandable and suitable for the selected user roles. Minor user interface improvements may still be completed before final deployment, especially for mobile responsiveness, clearer status labels and additional helper messages on important forms.

5.7 User Feedback

User feedback was considered important because ShortStay is used by different types of users with different technical knowledge. The main feedback areas are summarized below. These findings can be updated after collecting final questionnaire responses from real users.

Table 5.10 User Feedback Summary

Feedback Area	Positive Feedback	Suggested Improvement
User interface	Users found the layout clean and easy to understand	Improve mobile view and make some dashboard cards more compact
Property search	Search and filtering options help guests find suitable accommodation	Add map-based search and more sorting options
Booking flow	Booking process is clear when dates are available	Add stronger visual calendar indicators for unavailable dates
Host dashboard	Hosts can quickly understand earnings and booking information	Add downloadable monthly statements
Admin functions	Admin has useful control over users, properties and reports	Add advanced audit log filtering
Payment management	Payment status tracking is useful for financial control	Add more local payment gateway support such as PayHere
Inspection process	Field inspector workflow improves trust in property quality	Add mobile-friendly inspection image upload and checklist view

Chapter 6 – CONCLUSION

6.1 Introduction

This chapter concludes the ShortStay project by summarizing the implementation outcome, problems encountered during development, possible future improvements and lessons learnt. The ShortStay system was developed as a web-based marketplace for short-term accommodations. It provides a structured platform for guests to book accommodation and for hosts to publish and manage rental properties. The system also supports administrative control, payment management, inspections and reporting.

The project shows how a multi-role web application can be developed using React, Node.js, Express and MySQL. The final system includes the core modules required for an accommodation marketplace, such as user authentication, property listing management, booking management, payment processing, review handling, complaint handling and property verification.

6.2 Problems Encountered

Several technical and project-related problems were encountered during the development of the ShortStay system. These problems helped the team understand the complexity of building a real-world marketplace application.

- Handling multiple user roles was challenging because each role required separate dashboards, permissions and workflows. The system had to support guests, hosts, admins, payment managers and field inspectors without mixing their responsibilities.
- Designing secure authentication required additional care. Password hashing, JWT generation, token validation, account lockout and account suspension had to be implemented carefully to protect user accounts.
- Booking management was one of the most complex areas because the system needed to validate date ranges, calculate nights, check property availability and prevent double bookings.

- Payment handling required careful design because payment records must be linked correctly to bookings. Refunds, failed payments and payment reports also added complexity.
- Frontend and backend integration needed consistent API structures. Some frontend screens required data in a specific format, so API responses had to be designed clearly.
- File uploads for property, profile and inspection images required separate middleware and storage rules.
- Property verification required coordination between host, field inspector and admin workflows. This made the system more complex than a simple booking platform.
- Time management was difficult because the project included many modules and had to be completed within an academic schedule.
- Testing every role was time-consuming because each role has different screens and access permissions.

6.3 Future Improvements

Although the current ShortStay system includes the main marketplace features, several future improvements can be added to make the platform more complete, scalable and user-friendly.

- Develop a dedicated mobile application for Android and iOS users so that guests, hosts and inspectors can use the system more conveniently.
- Improve map-based property search by integrating advanced Google Maps features such as nearby places, distance calculation and location-based recommendations.
- Integrate local payment gateways such as PayHere to support Sri Lankan users more effectively, in addition to Stripe.
- Add multilingual support such as English, Sinhala and Tamil to make the platform easier for local users and tourists.

- Introduce AI-based property recommendations based on user search history, ratings, location and price range.
- Improve the availability calendar with clearer blocked dates, dynamic pricing and seasonal rates.
- Add SMS or WhatsApp notifications for booking confirmations, reminders, cancellations and payment updates.
- Enhance reporting with export options such as PDF and Excel reports for admins, hosts and payment managers.
- Add automated test suites for frontend and backend modules to improve long-term reliability.
- Deploy the system on a cloud platform with CI/CD pipeline, HTTPS, backups and monitoring.
- Improve performance using caching, image optimization and database indexing for large-scale usage.
- Add advanced dispute management and refund policies to handle guest-host issues more professionally.

6.4 Lessons Learnt

The ShortStay project provided practical experience in building a full-stack web application. It helped improve both technical and project-management skills.

- A clear requirement analysis is important before implementation because changes become more difficult after development begins.
- Role-based access control must be planned early when a system has different user types and permissions.
- Separating code into routes, controllers, models, middleware and utilities makes the backend easier to maintain.
- Reusable frontend components such as navigation bars, layouts and API helpers reduce duplicate code and keep the interface consistent.

- Database design is very important for booking systems because users, properties, bookings, payments, reviews and inspections are closely related.
- Testing should start from the early development stage instead of waiting until the full system is completed.
- Secure password handling, token validation and input validation are essential for protecting user data.
- Using Git and GitHub helps the team manage code changes and collaborate more effectively.
- Good documentation helps explain the system clearly to supervisors, users and future developers.
- User feedback is valuable because it highlights interface and workflow issues that developers may not notice during implementation.

In conclusion, the ShortStay system successfully demonstrates the development of a web-based short-term accommodation marketplace. The system provides a strong foundation for connecting guests and hosts while supporting administrative control, payments and property verification. With the suggested future improvements, the system can be developed further into a more complete commercial accommodation booking platform.

References

Appendix A - System Manual

System requirements

A.1 Introduction

The ShortStay system is a web-based online marketplace developed to connect guests with property owners for short-term accommodation bookings. The system provides a secure and user-friendly platform where guests can search and reserve accommodations, hosts can manage their property listings, administrators can monitor system activities, payment managers can handle financial transactions, and field inspectors can verify listed properties. This manual provides the necessary information to install, configure, and operate the system.

A.2 System Requirements

Hardware Requirements

Compon ent	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3	Intel Core i5 or above
RAM	4 GB	8 GB or above
Storage	10 GB Free Space	20 GB Free Space
Internet	Broadband Connection	High-Speed Internet

Software Requirements

Software	Version
Windows	Windows 10/11
Node.js	Latest LTS Version
MySQL Server	Version 8.0 or above
Visual Studio Code	Latest Version
Google Chrome	Latest Version
Git	Latest Version

A.3 Installation Procedure

Follow the steps below to install the ShortStay system.

1. Install Node.js on the computer.
2. Install MySQL Server and MySQL Workbench.
3. Install Visual Studio Code.
4. Clone the ShortStay project from the GitHub repository.
5. Open the project folder using Visual Studio Code.
6. Install all required project dependencies using:

`npm install`

7. Configure the environment variables in the `.env` file.
8. Import the MySQL database into the local server.
9. Start the backend server.

`npm start`

10. Start the frontend application.

`npm run dev`

11. Open the application using the provided local URL in a web browser.

A.4 Database Configuration

The ShortStay system uses MySQL as the relational database management system.

The following information should be configured before running the application.

- Database Name
- Database Username
- Database Password
- Database Host
- Database Port
- JWT Secret Key

A.5 Project Structure

The project is organized into two main sections.

- Frontend
- Backend

The frontend contains user interface components, pages, routing, and reusable components.

The backend contains controllers, routes, middleware, database models, authentication modules, payment processing, and API services.

A.6 Troubleshooting

Common issues that may occur during installation are listed below.

Problem	Solution
Database connection failed	Verify MySQL configuration and credentials.
npm install failed	Check internet connection and Node.js installation.
Application does not start	Ensure all required dependencies are installed.
Login unsuccessful	Verify database records and JWT configuration.

Appendix B - Design Documentation

B.1 Introduction

This appendix presents the design documentation used during the development of the ShortStay system. The design documents describe the overall architecture, database structure, object-oriented design, system workflows, and user interface layouts used to implement the proposed solution.

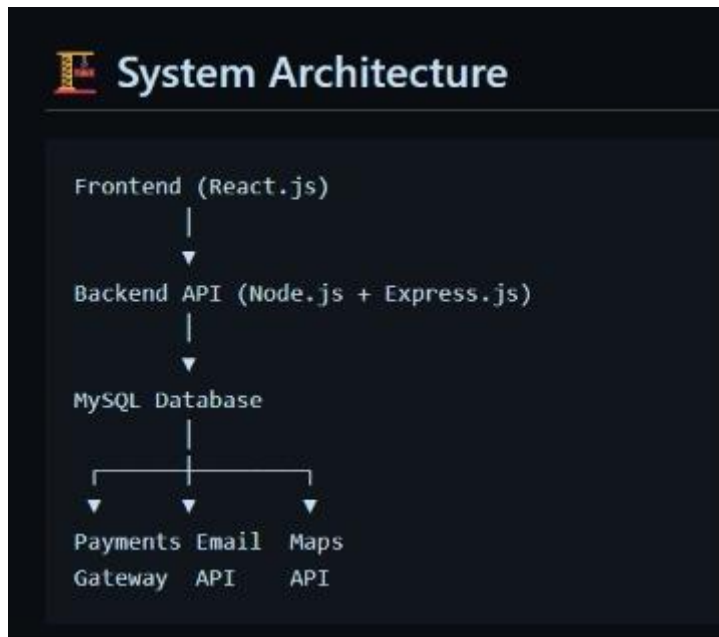
B.2 System Architecture

The ShortStay system follows a three-tier architecture consisting of the Presentation Layer, Business Logic Layer, and Data Layer.

The Presentation Layer provides interfaces for guests, hosts, administrators, payment managers, and field inspectors.

The Business Logic Layer processes user requests, validates system operations, manages authentication, booking management, property verification, and payment processing.

The Data Layer stores all application data in the MySQL database.



B.3 Use Case Diagram

The Use Case Diagram illustrates the interactions between users and the ShortStay system.

The main actors include:

- Guest
- Host
- Administrator
- Payment Manager
- Field Inspector

The major use cases include user registration, login, accommodation search, booking management, payment processing, property verification, report generation, and review management.

(Insert Use Case Diagram Here)

B.4 Class Diagram

The Class Diagram represents the object-oriented structure of the system, including classes, attributes, methods, and relationships among system entities.

Major classes include:

- User
- Guest
- Host
- Property
- Booking
- Payment
- Review
- Inspection

(Insert Class Diagram Here)

B.5 Entity Relationship Diagram

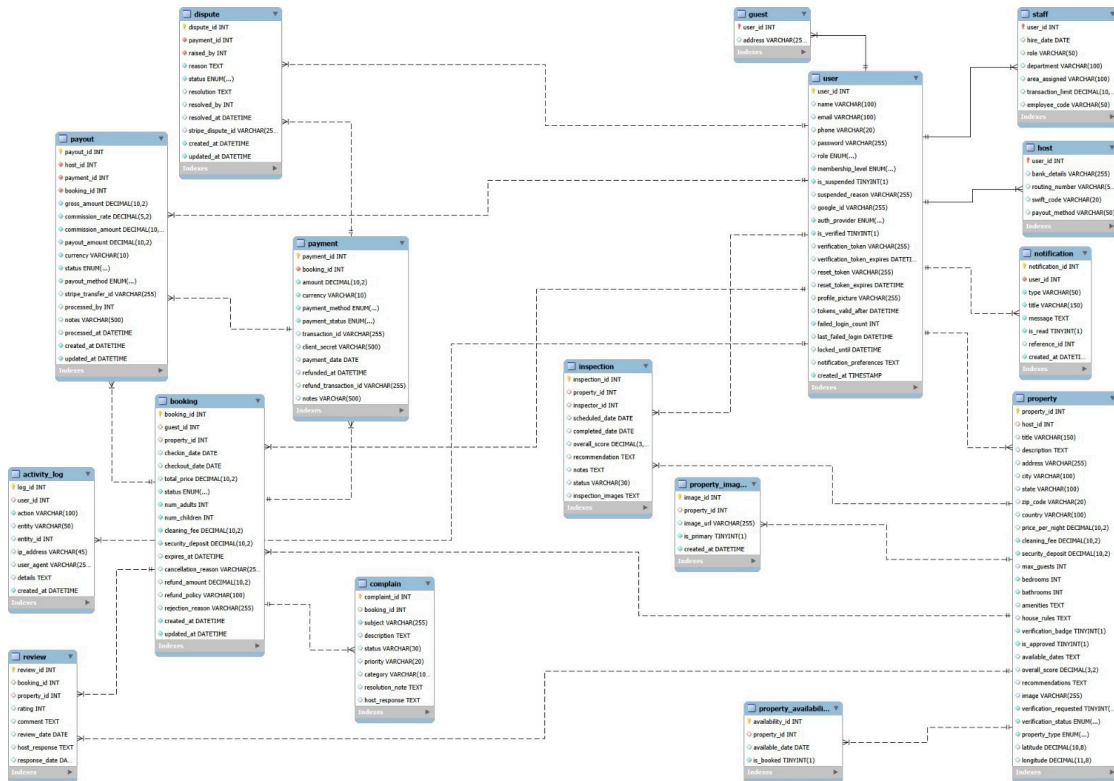
The ER Diagram illustrates the relationships between the database entities.

Major entities include:

- User
- Host
- Guest
- Staff
- Property
- Booking
- Payment
- Review
- Inspection
- Notification

- Complain

The relationships ensure data integrity and efficient management of accommodation bookings and payments.



B.6 Activity Diagrams

Activity diagrams describe the workflow of major system processes.

The following activity diagrams are included.

- User Registration
- User Login
- Property Listing
- Booking Process
- Payment Process
- Property Verification

(Insert Activity Diagrams Here)

B.7 User Interface Design

The user interface was designed to provide a responsive, user-friendly, and consistent experience across all user roles.

Major interfaces include:

- Home Page
- Login Page
- Registration Page
- Guest Dashboard
- Host Dashboard
- Administrator Dashboard
- Payment Manager Dashboard
- Field Inspector Dashboard

(Insert UI Screenshots or Figma Designs Here)

Appendix C - User Manual

C.1 Introduction

This user manual explains how different users can operate the ShortStay system. It provides step-by-step instructions for performing the major functions of the application.

C.2 User Registration

1. Open the ShortStay website.
2. Click the **Register** button.
3. Enter the required personal information.
4. Submit the registration form.
5. The user account will be created successfully.

(Insert Registration Screenshot Here)

C.3 User Login

1. Open the Login page.
2. Enter the registered email address.
3. Enter the password.
4. Click the **Login** button.
5. The user will be redirected to the appropriate dashboard.

(Insert Login Screenshot Here)

C.4 Searching for Properties

1. Navigate to the Browse Properties page.
2. Enter the preferred location.
3. Apply filters such as price, property type, and availability.
4. Select a property to view detailed information.

(Insert Search Page Screenshot Here)

C.5 Booking Accommodation

1. Open the selected property.
2. Select the check-in and check-out dates.
3. Confirm the booking details.
4. Proceed to the payment page.

(Insert Booking Screenshot Here)

C.6 Making Payments

1. Select the preferred payment method.
2. Enter the payment details.
3. Confirm the payment.
4. A booking confirmation message will be displayed.

(Insert Payment Screenshot Here)

C.7 Host Functions

The host can perform the following tasks:

- Add new properties
- Update property details

- Manage booking requests
- View earnings
- Track reservation status

(Insert Host Dashboard Screenshot Here)

C.8 Administrator Functions

The administrator is responsible for:

- Managing users
- Approving properties
- Monitoring system activities
- Managing complaints
- Generating reports

(Insert Administrator Dashboard Screenshot Here)

C.9 Payment Manager Functions

The Payment Manager can:

- View payment transactions
- Process refunds
- Generate payment reports
- Monitor financial records

(Insert Payment Manager Screenshot Here)

C.10 Field Inspector Functions

The Field Inspector is responsible for:

- Viewing inspection requests
- Conducting property inspections
- Approving or rejecting properties
- Updating inspection records

(Insert Inspector Dashboard Screenshot Here)

C.11 User Logout

1. Click the user profile icon.
2. Select the **Logout** option.
3. The system will terminate the current session and redirect the user to the login page.

(Insert Logout Screenshot Here)

Appendix D - Management Reports

Appendix E - Test Results

Test result for login and logout of the system

Test No	Test Description	Expected Result	Actual Result	Status
1				
2				
3				
4				

Test result for user creation of the system

Appendix F - Client Certificate

Glossary

Index

Group No:				
S.No.	Student Name	Registration Number	Index Number	Email ID
1.				
2.				
3.				
4.				
5.				
6.				
7.				
8.				
9.				
10.				